

⚠️ Technical Constraints & Optimization

To succeed within the constraints of public API quotas and to ensure a reproducible workflow, you must implement the following architectural patterns:

1. Resilient API Orchestration (Error 429 Handling)

Expect to encounter **Rate Limit Errors (HTTP 429)** while using the LLM free tiers.

- **Implementation:** All LLM-calling functions must include a **Wait & Retry** logic (e.g., exponential backoff).
- **Goal:** Your pipeline should be "fault-tolerant," meaning it handles temporary service interruptions without crashing the entire batch process.

2. Local Persistence & Caching

To optimize both cost and development speed, do not treat embeddings as transient data.

- **Storage Strategy:** Store all generated document embeddings and query vectors locally in **Parquet files** (or a local vector database).
- **Optimization:** Implement a check to see if a chunk has already been embedded before calling the API. This will prevent redundant computations, save significant processing time, and keep you within your daily quota.

Pro-Tip: Using **pandas** with **pyarrow** to manage these Parquet files is highly recommended for its efficiency with high-dimensional vector data.

You must implement ONE of the following projects, in a pair.

Project: Comparative Analysis of NLP Architectures for Automated Intent Classification

1. Project Overview

The objective of this project is to build, evaluate, and compare two distinct technical approaches for classifying customer banking queries. You will move from "classical" NLP (Word Embeddings + Supervised Learning) to "modern" Generative AI (Few-shot prompting + LLM-based classification). The project concludes with a business-case analysis focusing on cost, latency, and performance tradeoffs.

Dataset: [MTEB/Banking77](#)

This dataset contains 13,008 customer service queries across 77 fine-grained intent categories in the banking domain, split into training and testing datasets. Load the data using the [HF dataset interface](#).

Much of the code used here has been used in labs or in-class demos.

2. Technical Phase 1: The Classical ML Pipeline

You must develop a robust classification pipeline using traditional vectorization techniques.

- **Data Engineering:** Implement a preprocessing pipeline (text normalization, handling contractions, stop-word removal considerations).
 - **Feature Extraction:** Load a pretrained **Word2Vec** model to generate dense vector representations.
 - **Classification:** Implement and tune **one** of the following supervised learning algorithms:
 - K-Nearest Neighbors (KNN)
 - Random Forest
 - Support Vector Machine (SVM)
 - **Evaluation:** Report performance metrics (Accuracy, Precision, Recall, and F1-Score) on the held-out test set.
-

3. Technical Phase 2: The LLM Approach

You will leverage Large Language Models to perform the same classification task through prompt engineering and semantic search.

- **Embedding & Retrieval:** Use the gemini-2.5-flash model to vectorize the questions.
 - **Prompt Engineering:**
 - Design a system prompt that instructs the LLM to categorize tickets into the specific Banking77 labels.
 - **Constraint:** Prompt tuning must be performed exclusively on the **training set**. You must not use the test set for iterative prompt refinement.
 - **Optimization (Optional/Bonus):** Propose a strategy to improve performance using the training data (e.g., Retrieval-Augmented Generation (RAG) where similar labeled examples from the training set are injected into the prompt as "few-shot" examples).
-

4. Comparative Evaluation & Benchmark

You must conduct a rigorous comparison of the two systems developed in Phases 1 and 2.

A. Operational Benchmarking

Create a unified interface where a single ticket is passed to a function and the label is returned.

- **Latency Test:** Select 10 random tickets and measure the inference speed for both approaches.
- *Note: For LLM APIs, exclude network "retry" wait times to focus on processing speed.*

B. Performance & Qualitative Analysis

- **Metric Comparison:** Which system achieved a higher F1-Score on the test set?
 - **Complexity Audit:** Which system was more difficult to implement? Document the number of "moving parts" (infrastructure, dependencies, data cleaning steps).
 - **Error Analysis:** Conduct a qualitative review of "ambiguous" or "sarcastic" tickets. Which architecture handled nuance better?
-

5. Business & Production Case Study

Translate the technical findings into a professional recommendation.

- **Cost Estimation:** Calculate the projected monthly cost for **1 Million requests** based on the following infrastructure:
 1. **Classical ML:** Hosting on a **c4-standard-8** instance (GCP).

2. **LLM:** API costs for **Gemini Flash** combined with a smaller **c4-standard-2** instance for orchestration.
- **Latency Analysis:** Compare average response times and discuss the implications for a real-time user interface.
 - **Executive Decision:** You are the Lead Data Scientist for an online marketplace. Which solution do you choose for their support ticket system? Justify your choice by discussing the tradeoffs between **accuracy, cost, and maintainability**.
-

6. Deliverables

1. **Codebase:** Clean, documented Python code (Jupyter Notebooks or Scripts), in a dedicated github repo, set to private where you will invite [me](#).
2. **Technical Report:** A 3-4 page document detailing the methodology, hyperparameter choices, benchmark results, and the final business conclusion. Include it in the markdown format, inside your github repo as readme.md.

Project: Advanced Financial Intelligence with a RAG & LLM-as-a-Judge

1. Project Overview

The goal of this project is to build an intelligent retrieval system capable of extracting structured knowledge from complex financial documents (10-K filings, earnings reports). You will implement a "Naive RAG" (vector-only) and an automated evaluation framework using an **LLM-as-a-Judge** with a **Cross-Encoder** architecture.

2. Phase 1: The Core RAG Pipeline

You will first establish a baseline retrieval-augmented generation system.

- **Data Acquisition:** Download a set of annual financial reports (PDFs) for 3–5 publicly traded companies over a 3-year period. These will be provided in a zip along this file.
 - **Preprocessing:** Implement a cleaning pipeline to handle financial tables and specific document artifacts.
 - **Chunking & Embedding:** * Strategy: Fixed-size chunking of **500 tokens**.
 - Embedding: Generate vector representations for each chunk.
 - **Vector Search:** Build a similarity-based search index to retrieve the top k relevant chunks for a given financial query. The search index can be a python dictionary.
 - (OPTIONAL - easy) Include a Streamlit API to interact with your RAG using a web page.
-

3. Phase 2: Knowledge Extraction (GraphRAG Prequel)

In this phase, you will transition from unstructured text to a semi-structured knowledge base by extracting relationships.

Relationship Extraction

For every text chunk in the database, use an LLM (e.g., GPT-4o, Gemini Flash) to:

1. **Identify Entities:** Extract companies, executives, products, and financial metrics.
2. **Map Relationships:** Extract triplets in the form of (Entity A, Relationship, Entity B).
 - Example: (Apple Inc., COMPETES_WITH, Samsung Electronics)
3. **Metadata Storage:** Store these extracted relationships alongside the original text chunks in your database.

Competitive Intelligence & Evolution

- **Competitor Mining:** Use the LLM to identify companies explicitly mentioned as competitors within the documents.
- **Temporal Analysis:** Compare the identified competitors across multiple years.
 - *Analysis Question:* How does the competitive landscape evolve over time in these filings? (e.g., Are new tech startups entering the "Risk Factors" section?)

You can find inspiration in [LlamaIndex](#) prompt templating.

4. Phase 3: Generation & Advanced Evaluation

The final phase focuses on producing answers and measuring their quality using state-of-the-art evaluation techniques.

The RAG Engine

- **Contextual Generation:** Use the retrieved chunks (and their associated relationships) to generate a final answer to the user query.
- **Implementation:** You must prompt the LLM to prioritize the retrieved context and cite the specific document/page used.

Accuracy Testing

- **Benchmark:** Test the system using the [PatronusAI/FinanceBench](#) dataset. You will use the provided **Question-Answer pairs** but must use their own **RAG engine** to generate the context and response.

The LLM-as-a-Judge (Cross-Encoder)

You must implement a "Judge" LLM to automate the evaluation process.

- **Architecture:** Use a Cross-Encoder approach where the model receives the triplet: **(Query, Context, Generated Answer)**
 - **The Prompt:** Design a prompt for the Judge LLM to determine if the generated answer is a "good fit" based on the context.
 - **Output:** Binary (Yes/No) with a brief justification.
 - **Scoring:** Compute an overall **Accuracy Score** (number of "Yes" judgments / total questions) for the RAG engine.
-

5. Deliverables & Evaluation Criteria

- **Codebase:** Clean, documented Python code (Jupyter Notebooks or Scripts), in a dedicated github repo, set to private where you will invite [me](#).

- **Technical Report:** A 3-4 page document detailing the methodology, hyperparameter choices, benchmark results, and the final business conclusion. Include it in the markdown format, inside your github repo as readme.md.